Recursion

What is recursion?

Recursion is when a function calls itself to solve a smaller version of the same problem.

Imagine opening a box containing a smaller box, which contains another smaller box. You continue until you reach the final box, then work backwards.

Every recursive solution needs:

1. **Base case: when should the function stop?**
2. **Recursive case: how does it call itself?**
3. **Progress: how does each call move closer to the base case?**
4. **Return/unwinding: how do the waiting calls finish?**

Visualising a recursive call

`fact(4)`

`4 × fact(3)`

`4 × 3 × fact(2)`

`4 × 3 × 2 × fact(1)`

Base case: `fact(1) = 1`

The calls now return in reverse order, producing 1 → 2 → 6 → 24.

A function call stores information such as its local variables and return location on the call stack. If recursion never reaches a base case, the program may eventually run out of stack space.

Q1 Factorial

Factorial is defined as:

$$n! = n \times (n-1) \times \cdots \times 1$$

And $0! = 1$.

```
long long fact(int n) {
    if (n <= 1) {
        return 1;
    }

    return 1LL * n * fact(n - 1);
}
```

Dry run: `fact(4)`

Call	Operation
`fact(4)`	`4 * fact(3)`
`fact(3)`	`3 * fact(2)`
`fact(2)`	`2 * fact(1)`
`fact(1)`	Returns `1`

Now the answers return:

- `fact(2) = 2 × 1 = 2`
- `fact(3) = 3 × 2 = 6`
- `fact(4) = 4 × 6 = 24`

Complexity: Time $O(n)$, call-stack space $O(n)$.

Common mistake: calling `fact(n)` again with the same `n`. The recursive argument must get smaller.

Q2 — Fibonacci

The standard Fibonacci sequence is:

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

```
long long fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;

    return fib(n - 1) + fib(n - 2);
}
```

Dry run: `fib(4)`

$$fib(4) = fib(3) + fib(2)$$

$$= (fib(2) + fib(1)) + (fib(1) + fib(0))$$

$$= (1 + 1) + (1 + 0) = 3$$

The first values are 0, 1, 1, 2, 3, 5, 8, 13…

Q3 Sum of an array

Define `sumArray(a, n)` to mean the sum of the first `n` elements.

The key idea is:

- If `n == 0`, the sum is zero.
- Otherwise, add the last included element to the sum of the first `n-1` elements.

```
int sumArray(const vector<int>& a, int n) {
    if (n == 0) {
        return 0;
    }

    return a[n - 1] + sumArray(a, n - 1);
}
```

Example:

```
vector<int> a = {2, 4, 6};

cout << sumArray(a, (int)a.size());  // 12
```

Dry run

`sumArray([2,4,6], 3)`

$$= 6 + sumArray(a, 2)$$

$$= 6 + 4 + sumArray(a, 1)$$

$$= 6 + 4 + 2 + sumArray(a, 0)$$

$$= 6 + 4 + 2 + 0 = 12$$

Complexity: Time $O(n)$, stack space $O(n)$.

The `const vector<int>&` parameter avoids copying the entire vector in every recursive call.

Generate all subsequences

This is a key recursion topic because it introduces decision trees and backtracking.

A subsequence preserves the original order of selected elements, but the selected elements do not need to be adjacent.

For example, for `[1,2,3]`, `[1,3]` is a subsequence, but `[3,1]` is not.

For `[1,2]`, the subsequences are:

- — empty subsequence
- `[1]`
- `[2]`
- `[1,2]`

For an array of size n, there are 2^n subsequences, including the empty one.

Core idea: include or exclude

For each element, make two decisions:

1. Include the current element.
2. Exclude the current element.

```
void generate(int i,
              const vector<int>& a,
              vector<int>& path) {

    if (i == (int)a.size()) {
        cout << "[ ";
        for (int x : path) {
            cout << x << " ";
        }
        cout << "]\n";
        return;
    }

    // Choice 1: include a[i]
    path.push_back(a[i]);
    generate(i + 1, a, path);

    // Undo the choice
    path.pop_back();

    // Choice 2: exclude a[i]
    generate(i + 1, a, path);
}
int main() {
    vector<int> a = {1, 2, 3};
    vector<int> path;

    generate(0, a, path);
}
```

- `push_back(a[i])` includes the current element.
- The first recursive call explores all possibilities that include it.
- `pop_back()` removes that choice.
- The second recursive call explores possibilities that exclude it.

This process is called backtracking: make a choice, explore it, undo the choice, and explore the alternative.

Why is `pop_back()` necessary? Because the same vector is reused across branches. Without removing the previous choice, the next branch would inherit an incorrect state.

Complexity:

- Time: $O(n \cdot 2^n)$ when printing every subsequence.
- Auxiliary space: $O(n)$ for the recursion stack and current path.

Common student mistakes

- Printing at every recursive call instead of only at the base case.
- Forgetting `pop_back()`.
- Forgetting to increase `i`.
- Confusing a subsequence with a contiguous subarray.

9. Coin toss using recursion

For every coin toss, there are two possible outcomes: `H` or `T`.

For n tosses, the total number of possible sequences is:

$$2^n$$

For two tosses: `HH`, `HT`, `TH`, `TT`.

For three tosses: `HHH`, `HHT`, `HTH`, `HTT`, `THH`, `THT`, `TTH`, `TTT`.

C++ implementation

```
void coinToss(int n, string& path) {
    if ((int)path.size() == n) {
        cout << path << "\n";
        return;
    }

    // Choice 1: Heads
    path.push_back('H');
    coinToss(n, path);
    path.pop_back();

    // Choice 2: Tails
    path.push_back('T');
    coinToss(n, path);
    path.pop_back();
}
```

Call it:

```
int main() {
   string path;
   coinToss(3, path);
}
```

Explanation:

For every position, the function first explores the H branch, then the T branch. Once the string length reaches n, one complete outcome is printed.

Complexity:

- Time: $O(n \cdot 2^n)$ when printing every sequence.
- Recursion depth: $O(n)$.

For fair, independent coin tosses, every length-n sequence has probability $1/2^n$. The code above enumerates all possibilities; it does not simulate random tosses.

Various Recursion Patterns:

1. Linear recursion — one recursive call
Print numbers from N to 1.
Print numbers from 1 to N.
Factorial.

2. Recursive traversal — processing one element at a time
Find maximum/minimum in an array.
Check whether an array is sorted.
Reverse a string.

3. Multiple recursive calls — branching recursion
Fibonacci.
Generate all possible paths in a simple decision process.
Divide and conquer Algos

4. Include / Exclude — decision recursion
Generate all subsequences.
Generate all subsets.

5. Backtracking — choose, explore, undo

Generate permutations.
Combination Sum.
N-Queens.